FINOPS-03: DPS Consumption Optimization — When to Cut, Tune, or Filter

> **Series:** FINOPS — Cost Management & FinOps | **Reference:** 03 — DPS Consumption Optimization — When to Cut, Tune, or Filter | **Created:** May 2026 | **Last Updated:** 10/14/2026

Overview

FINOPS-01 told you what's being consumed. FINOPS-02 told you when an alert should fire. **FINOPS-03 answers the only question that matters after the alert fires: what do you actually change?**

Most cost-optimization conversations stall because the team treats every reduction option as roughly equivalent — *"let's just sample more,"* or *"let's just shorten retention."* They are not equivalent. Each option has a different blast radius, a different reversibility profile, and a different relationship to the observability the business actually needs. Picking the wrong lever produces savings that erode the value of the platform.

The framework: three levers — Cut, Tune, Filter. Cut drops a signal entirely at the source. Tune adjusts a parameter on an otherwise-flowing signal (sample rate, retention period, resolution). Filter drops a *subset* of a flowing signal conditionally (specific severities, specific paths, specific apps). Each lever applies in different situations; mixing them up is the most common failure mode.

Bias toward upstream levers. The further upstream a change happens, the cheaper it is and the more reversible. Cut at the source (don't emit) beats Filter at OpenPipeline (don't ingest) beats Tune at storage (ingest cheaply). A flowing log stream that's been DEBUG-filtered at OpenPipeline still cost something to transit; the same stream that never emitted DEBUG didn't.

> **Scope:** Dynatrace SaaS on DPS. The decision framework is platform-agnostic but the implementation handles (OpenPipeline filter syntax, retention configuration, sampling controls) are Dynatrace-specific. Implementation depth lives in cross-referenced series (ORGNZ for buckets and retention; OPLOGS for sampling; OPIPE for non-log pipeline shapes).

Table of Contents

1. Short Answer
2. The Three Levers — Cut / Tune / Filter
3. When to Cut
4. When to Tune
5. When to Filter
6. Decision Tree by Data Type
7. Trade-offs and Hidden Costs
8. Worked Example — Production Log Stream
9. Worked Example — High-Cardinality Metric
10. Cross-References to Implementation Depth
11. Recommended Approach
12. Summary and Next Steps

Prerequisites

Requirement	Details
Familiarity with FINOPS-01	Understanding of per-capability schema and the `dt.system.events` / `dt.billing.*` surfaces — this entry assumes the *what is being consumed* picture is already in place.
Familiarity with FINOPS-02	This entry is the *response* layer to the alerts FINOPS-02 sets up.

Requirement	Details
Audience	Platform / Observability Lead (primary); Executive / Procurement (TL;DR, §7 trade-offs, §11 recommendation).
Companion series	ORGNZ (bucket strategy and retention), OPLOGS (OpenPipeline filtering and sampling), OPIPE (OpenPipeline for non-log signals), ADOPT-05 (optimization in the maturity-roadmap context).
Posture	This is a decision framework, not a tuning recipe. Decisions belong to the team that owns the affected workload; this notebook structures the conversation.

1. Short Answer

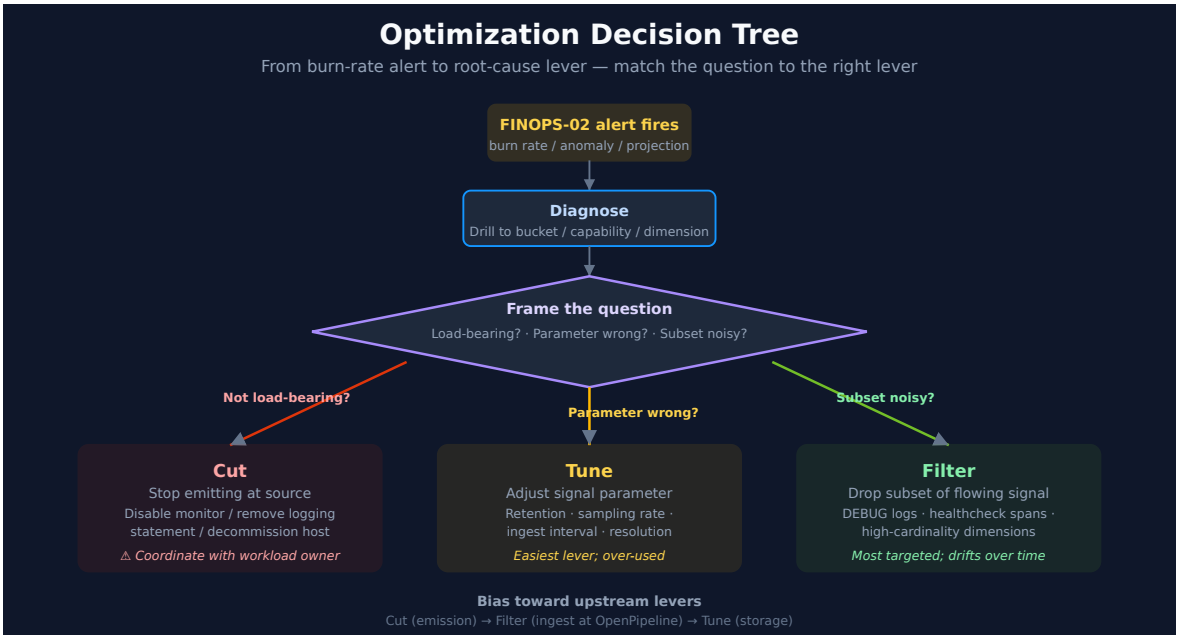
Lever	What it does	When to reach for it	Reversibility
Cut	Stop emitting the signal at the source (agent setting, app config, integration disabled)	The signal has no consumer that depends on it; or the cost-of-emission consistently exceeds the value	High — turn the signal back on
Tune	Keep the signal flowing but adjust a parameter (sample rate, retention, metric resolution)	The signal is useful but at a different fidelity than current; rate / depth can be reduced without losing the workload	Medium — re-tune the parameter
Filter	Drop a subset of an otherwise-flowing signal at OpenPipeline (or equivalent)	The signal is useful overall but specific subsets are noise (DEBUG logs, healthcheck spans, synthetic-test bots)	High — remove the filter

The three-lever split is this entry's framing, not a Dynatrace taxonomy — Dynatrace's own Optimize guide names two levers, sending and keeping less data and querying it more efficiently. Cut, Tune, and Filter subdivide the first; query-side tuning (§4) covers the second.

The headline: match the lever to the question. *"Is this signal still load-bearing?"* → Cut. *"Is the granularity right?"* → Tune. *"Is some-but-not-all of it noise?"* → Filter. Reach for the upstream-most option that solves the problem — emission-time changes are cheaper than ingest-time, which are cheaper than storage-time.

> **Sources:** Optimize (DT docs) — *"Two levers drive DPS cost down: Send and keep less data. Query your data more efficiently with DQL, dashboards, workflows."* Best practices for optimizing metrics cost (DT docs) — *"Generally, the closer to the client you drop data, the lower the overall cost."* DPS Log Management (DT docs) — *"filtering generates usage for Ingest & Process, but not for Retain"*.

2. The Three Levers — Cut / Tune / Filter



This entry sorts every consumption-reduction option into one of three patterns. Naming them helps the conversation move past "reduce cost" to deciding which signal-emission behavior to change.

Cut

Stop emitting the signal at the source. Examples:

- Disable a synthetic monitor that no one looks at
- Remove a logging statement from application code
- Disable verbose collector exporters
- Disable a Dynatrace integration that no team owns
- Decommission a host that no longer hosts production workload

Cut is the strongest lever — it eliminates the underlying activity, not just its observability. It is also the lever with the most organizational friction, because *someone* originally chose to emit the signal and someone has to confirm it's safe to stop.

Tune

Keep the signal flowing but adjust a parameter. Examples:

- Reduce log retention from 35 days to 14 days (Logs - Retain)
- Lower trace sampling rate (Traces - Ingest)
- Increase metric ingestion interval beyond one minute (Metrics - Ingest — sub-minute collection is already billed as at most one data point per minute)
- Move a bucket from Retain with Included Queries to standard Retain
- Switch a synthetic monitor from 1-minute to 5-minute frequency
- Slow a dashboard's auto-refresh from 1 minute to 5 minutes (Logs / Events / Traces - Query)
- Narrow a dashboard tile's default time range from 30 days to 24 hours
- Reduce session retention for a RUM application

Note that the last three examples are **query-side**, not ingest- or storage-side: Logs, Events, and Traces bill on query execution as well as on ingest and retention (Digital Experience Monitoring data has its own query billing event too — FINOPS-01 §9), so how often a dashboard re-reads its data is a tunable parameter in exactly the same sense as retention. § 4 covers this surface in full. Tune preserves the signal's existence — useful for cases where the signal is load-bearing but at lower fidelity than current. In community practice, Tune is the lever most often over-used because it requires no organizational conversation: the platform team can lower retention without consulting any application team.

Filter

Drop a subset of an otherwise-flowing signal, typically at OpenPipeline (logs / events / spans) or at the source SDK (custom metrics).

Examples:

- OpenPipeline pipeline that drops DEBUG / TRACE level logs in production buckets
- Span filter that excludes spans from healthcheck endpoints
- Metric filter that excludes test / synthetic-traffic dimensions

- Log filter that drops noisy known-benign error patterns (e.g., expected 404s on /favicon.ico)

Filter is the most targeted lever — keep the load-bearing subset, drop the noise. In community practice it is also the lever most likely to drift over time: a filter that was correct in 2024 may suppress a signal that became load-bearing in 2026. Filters need ownership and periodic review.

> **Sources:** OpenPipeline (DT docs), Optimize (DT docs), Best practices for optimizing metrics cost (DT docs) — *"each timeseries is billed for no more than one data point per minute."* Examples are illustrative; the underlying mechanisms are documented per-capability.

3. When to Cut

Cut applies when the signal has no consumer that justifies the cost-of-emission. In community practice, three diagnostic questions settle it:

Q1 — Who looks at this signal?

If the answer is "nobody for the last 90 days," the signal is a candidate. The audit pattern: query notebook open events, dashboard queries, and alert configurations that reference the signal. If none exist over a 90-day window, the signal is operationally dark.

Counter-question: is the signal a *compliance* requirement (audit logs, security events, regulator-mandated retention)? Compliance retention is not a value question — the signal must exist for the contract / regulation even if no one looks at it. Do not Cut compliance signals; Tune retention to the minimum mandated period instead — and verify that period against your specific regulatory framework.

Q2 — Does any alert / SLO depend on this signal?

If the signal feeds a workflow, an SLO calculation, a Davis problem-detection input, or a Cost Monitor, it is load-bearing in a way that's hard to see by reading logs / dashboards. Cross-check alert configurations before recommending Cut.

Q3 — Has this signal generated actionable insight in the last incident?

An audit by incident — for the last 5 production incidents, did this signal contribute to detection, diagnosis, or post-mortem? If consistently no, the signal is operationally low-value and Cut is appropriate.

How to Cut safely

1. **Communicate before disabling.** Find the team that owns the workload emitting the signal, even if they're not the team consuming it. The emit-side team may have a non-obvious dependency.
2. **Stage the Cut.** Disable in a non-production environment first, observe for one week, then promote to production.
3. **Document the Cut.** A short note in the team's runbook on what was disabled, when, and why. If the signal needs to come back, the documentation tells the next operator how.
4. **Set a review date.** Cuts that were correct in 2024 may be wrong in 2026 — schedule a 6-month review of the disabled signals list.

4. When to Tune

Tune applies when the signal is load-bearing but at lower fidelity than the current configuration. Three categories of tuning:

Retention tuning

In community practice, the most common Tune: logs / spans / events stored for 35 days when the use case only references the last 7 days. Retention is set per bucket and can be re-configured at any time. The workflow that works:

1. Audit query history: what's the longest lookback that actually fired against this bucket in the last 90 days?
2. Add a buffer — typically 1.5-2× the longest observed lookback.
3. Verify the buffer covers known periodic patterns: month-end batch jobs, quarterly audits, annual cleanups.

4. Apply the new retention; observe for one full periodic cycle (often a month) before declaring success.

Retention tuning is the easiest lever to justify because it has no impact on emission, ingestion, or current investigation — only on historical lookback that wasn't being used anyway. It is the lever to reach for first when in doubt.

Sampling tuning

For traces especially. Reduce trace-ingest sample rate from 100% (all spans) to a lower rate (e.g., 25% — keep 1 in 4 traces). Workflow:

1. Determine the *statistical* requirement: at what sampling rate do you still detect error patterns reliably? (In community practice, 10-25% is often enough for apps with sustained traffic — verify against your own error rates.)
2. Prefer head-based sampling (deterministic per-trace) where it meets the need — it is simpler to reason about.
3. Keep error and exception traces at a higher rate where your sampling mechanism supports it (for example, tail-based sampling in an OpenTelemetry Collector) — confirm what your ingest path offers before relying on it.
4. Measure investigation success after one month — can your team still root-cause issues with the new sample rate?

Resolution tuning

For metrics. Lengthening a metric's collection interval lowers `data_points` consumption — **but only above one minute**. Dynatrace aggregates sub-minute measurements into one-minute buckets and bills each timeseries for at most one data point per minute, so moving a metric from 10-second to 1-minute collection changes nothing on the bill, while moving it from 1-minute to 5-minute cuts its billed points ~5×. Workflow:

1. What SLO depends on this metric? If it's a high-criticality SLO (sub-minute MTTD), keep one-minute resolution.
2. Otherwise, consider 5-minute or longer — Dynatrace names less-critical metrics, metrics read over hours or days, and slow-changing metrics such as disk usage as candidates. Longer intervals leave gaps at one-minute resolution, which can affect existing alerts.
3. Apply per-metric, not globally — some metrics warrant high resolution; most do not.

Query-side tuning

The three tuning categories above are all ingest- or storage-side. **Logs, Events, and Traces also bill on query execution** — a Query capability separate from Ingest and Retain (FINOPS-01 §9 documents the schema; `client.source` — a dashboard URL for dashboard tiles — attributes the scan to a dashboard rather than a person). That makes query volume a fourth Tune surface — Dynatrace's own guidance is that with log queries *you are charged per execution* of every auto-refresh. In community practice, on a tenant with a wall-mounted dashboard estate it is often the largest one nobody is looking at. Note the asymmetry: metric queries through the `timeseries` command are always included and never billed, so this lever is about dashboards and notebooks reading **logs, events, and traces**, not metric tiles. A `makeTimeseries` over logs is a log query, not a metric query, and bills like one.

Three dials, in the order worth pulling them:

1. Refresh cadence. A dashboard on a 1-minute auto-refresh runs every one of its tile queries roughly **1,440 times a day**. Moving it to 5 minutes cuts that by ~5× and changes nothing about what the dashboard *shows* — the same tiles, the same queries, the same data, just re-read less often. It is among the cheapest consumption reductions in the platform because it costs no fidelity at all. DASH-02 §6 has the per-audience cadence table (NOC wall 1-2 min, on-call 2-5 min, executive wall 10-15 min, investigation manual); the FinOps read on that table is that anything faster than its audience needs is pure waste. Start by finding dashboards refreshing faster than their tier warrants.

Caveat — log buckets on Retain with Included Queries. Customers who choose that bucket option *"are not charged for those queries executed within the scope of the Included Queries retention period"*; only the logs a query reads outside that period bill under Log - Query. A dashboard whose log tiles read only such a bucket, inside that period, costs the same at any refresh cadence — check which bucket and time range a tile reads before tuning its cadence for cost. Cadence still matters for event and trace tiles, and for log tiles on buckets without the option.

2. Tile default time ranges. A tile whose default range is 30 days scans 30 days on every refresh, even when every viewer immediately narrows to the last hour. Multiply by cadence and this dominates. Set each tile default to the range the tile is actually *read* at, and let viewers widen deliberately.

3. Dashboard count. Duplicate near-identical dashboards multiply query volume by the number of copies. Consolidating five team-specific clones behind one `$team` variable is a **Cut**, not a Tune — it removes the queries rather than adjusting them — and it belongs in

§3 with the other Cut decisions. It appears here because it is usually discovered while auditing cadence.

> **Rolling out (SaaS 1.344): dashboard queries now stop automatically when the browser tab closes.** Published 07/27/2026, staged tenant rollout from 07/29/2026 — verify whether it has reached your tenant. This removes a small, hard-to-attribute slice of consumption from queries that outlived the tab that started them. **It is not a lever you operate**, and it does not change the arithmetic above: an **open** tab still refreshes on its interval and still bills for every scan. Cadence and tile scope remain the load-bearing controls; treat 1.344 as removing an accounting nuisance, not as a substitute for tuning.

Query-side tuning has one property that makes it unusually easy to sell: unlike retention or sampling, it is **fully reversible and costs no fidelity**. If a team discovers that 5 minutes is too slow for their incident workflow, put it back. Nothing was discarded.

When NOT to Tune

- **Compliance-mandated retention:** tune *down to the minimum*, not below.
- **Security-investigation logs:** under-sampling here is paid for during the next breach investigation. Sampling decisions on security signals merit security-team sign-off.
- **SLO-feeder metrics:** if the metric is the foundation of an availability SLO, reducing resolution may break the SLO calculation. Verify before tuning.
- **The dashboard that is your detection surface:** if a NOC wall dashboard is how an incident is actually noticed, slowing its refresh trades money for detection time. Tune cadence on the dashboards nobody watches in real time first, and leave the ones on the wall alone.

> **Sources:**
>
> - Best practices for Log Management and Analytics (DT docs) — per-bucket retention periods
> - Best practices for optimizing metrics cost (DT docs) — *"For metrics collected at intervals shorter than one minute, Dynatrace aggregates the incoming measurements into fixed one-minute buckets for efficient storage and querying. Therefore, each timeseries is billed for no more than one data point per minute."*; *"Longer ingest intervals can introduce gaps when you view data at one-minute resolution. This can impact existing alerting setups."*
> - Metrics powered by Grail - Query (DT docs) — *"is always included in your DPS subscription and*

> *never results in costs."*; *"Queries involving other data types generally incur usage at each query, even when they output timeseries format."*
>
> - Optimize dashboards running log queries (DT docs) — *"If you are using log queries, you are charged per execution."*
> - Log Analytics overview (DT docs) — *"Customers who select the Retain with Included Queries option in the bucket configuration are not charged for those queries executed within the scope of the Included Queries retention period."*; *"only these logs outside of the Retain with Included Queries are billed individually with the usage based Log - Query model."*

5. When to Filter

Filter applies when the signal is useful overall but specific subsets are noise. The most common filter targets:

Logs — severity-based filters

DEBUG / TRACE logs in production. Dynatrace lists *debug-level logging left on in production* among the typical causes of an ingest spike, and in community practice it is the highest-impact filter. Logs can be filtered on ingest with OneAgent log ingest rules or OpenPipeline — sent to a different bucket or dropped outright — conditionally on app or environment.

Implementation: OpenPipeline pipeline with a drop processor whose matching condition combines `in(loglevel, {"DEBUG", "TRACE"})` with a predicate that scopes it to production sources (for example a host-group or Kubernetes-namespace field). Full syntax covered in OPLOGS.

Common mistake: dropping ALL DEBUG everywhere. Some pre-production environments need DEBUG; some pre-incident-replay scenarios need DEBUG. Filter on environment, not globally.

Spans — endpoint-based filters

Healthcheck endpoints (`/health` , `/ping` , `/readiness` , `/liveness`) emit one span per check, often every few seconds, and almost never carry diagnostic value. A trace filter that drops these is high-impact, low-risk.

Common mistake: dropping spans from any endpoint matching `/health` — some apps have legitimate `/health/<subsystem>` endpoints that are diagnostically valuable. Be specific.

Logs — pattern-based filters for known-benign noise

Repetitive log lines that are *known to be benign* (e.g., favicon.ico 404s, robots.txt 404s, expected timeout messages from a known-flaky third-party). Drop them; they were never going to be investigated.

Common mistake: filtering on broad patterns. *"Drop all 404s"* hides the day a real customer flow starts 404-ing. Filter on the specific path or message pattern, not the category.

Metrics — high-cardinality dimension filters

Custom metrics emitted with a high-cardinality dimension (user.id, request.id, span.id as a label) produce one time series per unique value, and every series that sends data points is billed for them. Filter the dimension at the SDK or at ingest — an OpenPipeline DQL or Remove-fields processor — keep the metric, drop the high-cardinality label.

Common mistake: assuming the dimension is the data. Often, the metric is useful at aggregate (counts per minute) and the dimension was added for debug-style exploration. Confirm before dropping the dimension.

When to Filter vs Tune

In this entry's framing, Filter targets a *known-noisy subset*. Tune adjusts a *parameter applied to everything*. If the noise is concentrated in a subset you can name ("DEBUG logs in prod," "healthcheck spans," "synthetic-traffic dimension on app metric X"), Filter. If the noise is *diffuse* ("all our traces are too detailed for the value they provide"), Tune the sample rate.

Filter ownership and review

In community practice, filters drift: a rule that was correct in 2024 may suppress a signal that became load-bearing in 2026. Every filter needs:

- An owner (the team responsible for the filtered signal)
- A rationale (what makes the filtered subset noise)

- A review date (6-12 months)

Filters without owners become invisible cost optimization that no one can safely change.

> **Sources:**
>
> - Trace a cost spike to its root cause (DT docs) — typical ingest-spike causes include *"debug-level logging left on in production"*
> - Best practices for Log Management and Analytics (DT docs) — filter logs on ingest with OneAgent or OpenPipeline
> - Best practices for optimizing metrics cost (DT docs) — *"High-cardinality dimensions, such as user IDs or request IDs, significantly increase metric storage requirements. Removing a high-cardinality dimension reduces the number of stored timeseries and can lower cost."*
> - OpenPipeline (DT docs), OPLOGS topic series — full filter-syntax depth

6. Decision Tree by Data Type

A working version of this entry's framework for each major data type — verify each recommendation against your tenant's actual usage patterns before applying it:

Logs

1. **Cut first?** Is the log-emitting application code still relevant? If a service emits logs no one reads and the service itself is on a deprecation path, Cut at the application.
2. **Filter next?** Are DEBUG / TRACE logs being emitted in production? Filter at OpenPipeline. Are known-benign patterns repeating (favicon 404s, etc.)? Filter pattern-by-pattern.
3. **Tune last?** Adjust retention per bucket based on actual lookback patterns. Move buckets between Retain and Retain-with-Included-Queries tiers based on query frequency.

Traces

1. **Cut first?** Is the service still in active use? If not, OneAgent / OTel SDK can be disabled for the service entirely.

2. **Filter next?** Are healthcheck spans dominating volume? Filter them at the OpenPipeline pipeline. Are synthetic-test traces inflating ingest? Filter them.
3. **Tune last?** Reduce overall sample rate, keeping error traces at higher rate (differential sampling).

Metrics

1. **Cut first?** Is the custom metric still in use? Audit Notebooks, Dashboards, and alerts.
2. **Filter next?** Are high-cardinality dimensions inflating data_points without value? Drop the dimension at the SDK.
3. **Tune last?** Lengthen the ingest interval for non-SLO-feeder metrics beyond one minute (e.g., 1 min → 5 min). Going from 10 s to 1 min saves nothing — sub-minute collection is already billed as one data point per minute.

Synthetic monitors

1. **Cut first?** Is anyone watching this monitor? Disable monitors no one has touched in 90 days.
2. **Tune next?** Reduce frequency (5min → 15min) for monitors that don't feed sub-15-minute SLOs.
3. **Filter** — N/A for synthetic; synthetic actions are atomic, no subset to filter.

Real User Monitoring (RUM)

RUM bills **per session** (`billed_sessions` — FINOPS-01 §7), so the unit of waste is a session nobody looks at.

1. **Cut first?** Is anyone watching this RUM application? An instrumented application with no dashboard, no SLO, and no owner is pure cost — every session it produces is billed and none is read. This is the highest-yield RUM lever and the one most often missed, because RUM instrumentation is usually added once and never revisited. Audit applications against actual dashboard, SLO, and alert references before assuming any of them is load-bearing.
2. **Tune next?** Session retention. Apply the same discipline as log retention in §4 — audit the longest lookback that actually fired against session data in the last 90 days, then add a 1.5–2× buffer, then verify the buffer covers known periodic patterns before applying. **`userSessionRetention` was added to the environment storage configuration in API 1.344** (published 07/15/2026, staged rollout from 07/29/2026) —

verify the field exists on your tenant before scripting against it. Until it lands, set session retention through the environment storage settings UI; that path works today and is unaffected by the API addition.

3. **Filter last?** Session Replay sampling is the main subset dial — replay is the expensive part of a session, and a sampled replay rate usually preserves the investigative value. Excluding specific user actions (health-probe clicks, synthetic-traffic paths, internal admin routes) trims the rest.

Dashboards and other query volume

Cross-cutting rather than a data type — this is the consumption caused by *reading* logs, events, and traces, and Digital Experience Monitoring data, which has its own query billing event (see the query-side tuning subsection in §4). Metric queries through `timeseries` are always included and never billed, so metric-only dashboards are out of scope here.

1. **Cut first?** Duplicate and abandoned dashboards. Five team-specific clones of one dashboard run five times the queries; consolidate them behind a `$team` variable. A dashboard nobody has opened in 90 days that is still auto-refreshing on a wall screen is the pure-waste case.
2. **Tune next?** Refresh cadence, then tile default time ranges. A 1-minute refresh runs each tile query ~1,440×/day; 5 minutes cuts that ~5× with no change to what the dashboard shows. Match cadence to audience using the DASH-02 §6 table. Log tiles reading a Retain with Included Queries bucket inside its included period are not billed per query, so cadence is not a cost lever for them (§4).
3. **Filter** — the query equivalent of a filter is scoping the tile query itself: narrow the bucket, add the filter earlier in the pipeline, drop fields you do not render. This is DQL optimization rather than a consumption lever per se, but it reduces bytes scanned on every one of those 1,440 executions.

Host monitoring (Full-Stack / Infrastructure)

1. **Cut first?** Decommission hosts that no longer host workload. Downgrade Full-Stack to Infrastructure for hosts that don't need code-level visibility.
2. **Tune** — no granularity dial for host monitoring; the lever is monitoring mode (Full-Stack / Infrastructure / Foundation).
3. **Filter** — N/A; host monitoring is per-host billed.

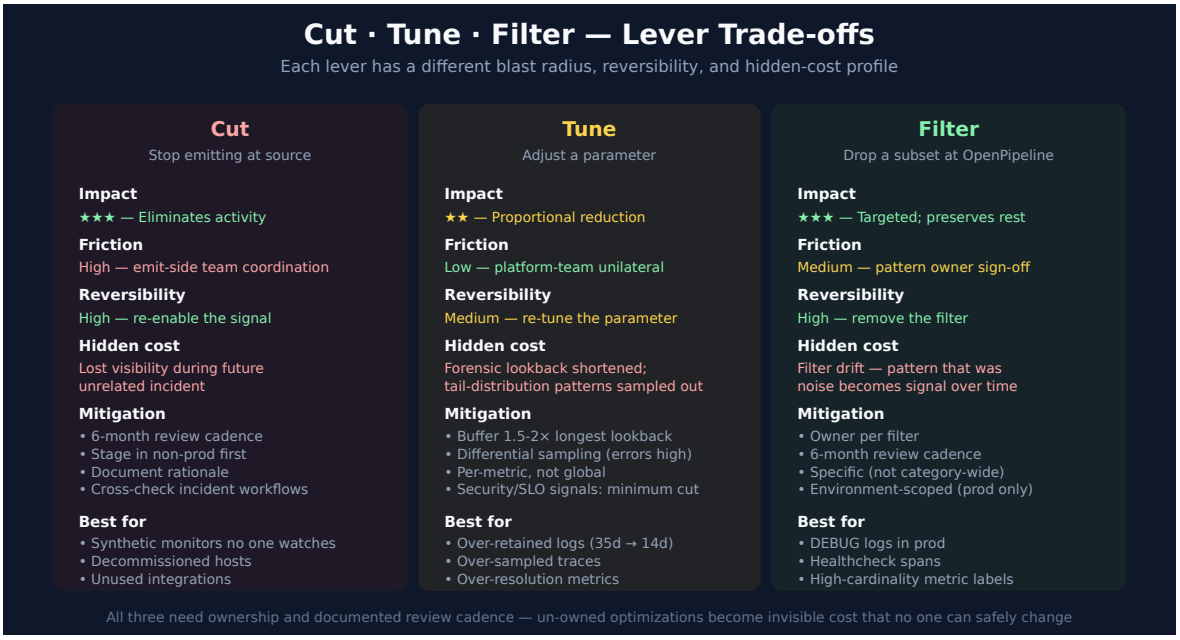
Automation Workflows

Automation Workflow bills **workflow-hours**: every deployed standard workflow is billed for each hour it exists, run or not (FINOPS-01 §7).

1. **Cut first?** Delete standard workflows nobody uses — they bill every hour they exist, whether they ran in the last 90 days or not.
2. **Tune next?** Where a workflow qualifies, make it a simple workflow, and leave unfinished work as a draft — neither consumes workflow-hours. **Reducing schedule frequency does not reduce Automation Workflow consumption**; it only reduces the Standard Function Calls and query usage of the tasks each run executes.
3. **Filter** — workflow filtering is at the trigger condition (only fire if X), which is an action-level filter. Like schedule frequency, it trims the AppEngine and query usage of the runs, not the workflow-hours.

> **Sources:** Per-capability billing semantics: DPS docs. Calculate your consumption of Automation Workflow (DT docs) — *"Standard workflows consume workflow hours."*; *"Simple workflows do not directly consume workflow hours"*; *"Draft-only workflows do not directly consume workflow hours"*; *"Each workflow execution triggers Standard Function Calls that execute tasks"* Metrics powered by Grail - Query (DT docs) — *"is always included in your DPS subscription and never results in costs."* Best practices for optimizing metrics cost (DT docs) — *"each timeseries is billed for no more than one data point per minute."*

7. Trade-offs and Hidden Costs



Lever	Visible benefit	Hidden cost	Mitigation
Cut	Largest cost reduction; eliminates underlying activity	Lost visibility may surface during an unrelated future incident — "I wish we still had X"	6-month review cadence; documented Cut rationale; non-prod staging before prod
Tune (retention)	Easy to apply; no emit-side coordination	Forensic / audit lookback becomes shorter — the next breach investigation may exceed the new retention window	Audit longest *historical* lookback used; buffer 1.5-2×; coordinate with security on logs of audit interest
Tune (sampling)	Reduces trace volume proportionally	Tail-of-distribution patterns (rare errors, edge-case latency) may sample out	Keep error / exception traces at higher rate (differential sampling); review investigation-success metrics after first month
Tune (resolution)	Reduces metric data_points	SLO calculations with sub-resolution sensitivity may break or become noisy	Per-metric, not global; preserve high resolution for SLO-feeder metrics

Lever	Visible benefit	Hidden cost	Mitigation
Filter (severity)	Drops known-noisy log volume	A real DEBUG-level diagnostic might be needed during a future deep-investigation	Filter on environment (prod only), not globally; keep DEBUG flowing in pre-prod
Filter (endpoint)	Drops healthcheck-span volume	If a healthcheck endpoint starts misbehaving, the noise *was* the signal	Filter on specific paths, not patterns; monitor health endpoints at the underlying-app layer instead
Filter (pattern)	Drops known-benign error patterns	The known-benign pattern may shift to known-malign without a clear signal	Pattern-specific, not category-wide; document the rationale per filter
Filter (cardinality)	Reduces metric series explosion	The cardinality may have been load-bearing for a specific debug workflow	Confirm with the team that added the high-cardinality dimension; preserve the dimension on a subset if needed

The category of trade-off that hurts most

Coverage gaps surface at the worst moment. A Cut to a synthetic monitor that no one watches is fine — until the day the corresponding service has a real outage and the absent monitor would have caught it 20 minutes earlier. In community practice, the cost-savings math from optimization tends to understate this risk by treating signals as if their value were proportional to their use-frequency. Signal value is observed to be highly skewed — most signals are operationally dark most of the time and load-bearing during specific incidents. The exact distribution depends on your workload and incident patterns; verify it in your own environment.

Mitigation: never optimize a signal that participates in incident-response workflows without explicit incident-team sign-off. *"Did the last 3 incidents reference this signal?"* is a better question than *"is anyone querying this dashboard regularly?"*

8. Worked Example — Production Log Stream

Scenario: The FINOPS-02 weekly burn-rate alert fires on Monday morning. The `payments-prod` bucket grew 60% week-over-week. The platform team wants to act before the next Cost Monitor anomaly.

Step 1 — Diagnose

Run the FINOPS-01 §6 query to confirm the bucket-level numbers. Then drill in:

```
// Diagnose the spike — payments-prod log volume by loglevel over the last 7 days
fetch logs, from:-7d
| filter dt.system.bucket == "payments-prod"
| summarize { record_count = count() }, by:{ loglevel }
| sort record_count desc
```

Suppose the result shows DEBUG = 14M records, INFO = 800K, WARN = 50K, ERROR = 5K. DEBUG is overwhelmingly dominant.

Step 2 — Frame the levers

- **Cut?** The payments service is in active use — its logs are load-bearing. Cut is not the right lever.
- **Tune?** Retention is already 14 days. Lowering retention won't help the ingest spike.
- **Filter?** DEBUG logs in production are the canonical Filter target — Dynatrace's own spike tutorial names debug logging left on in production as a typical ingest-spike cause. The application team may have left a feature-flag DEBUG enabled in a recent deploy, or DEBUG is on permanently and shouldn't be.

Filter is the right lever. Specifically, an OpenPipeline pipeline that drops `loglevel == "DEBUG"` records bound for the `payments-prod` bucket.

Step 3 — Coordinate before filtering

Ping the payments team before adding the filter. Two outcomes possible:

1. **Team confirms DEBUG should not be on in prod:** they fix the application config; the ingest returns to baseline; no filter needed. (Best outcome — root-cause fix.)

2. **Team explains DEBUG was intentionally enabled for an ongoing investigation:** add the filter with a time-boxed expiry (e.g., 30 days), and notify the team when the filter is removed.

Step 4 — Implement (after coordination)

Implementation depth lives in OPLOGS. The filter spec at the framework level:

- Scope: records bound for the `payments-prod` bucket
- Condition: `in(loglevel, {"DEBUG", "TRACE"})`
- Action: drop
- Owner: platform team (with payments team as informed party)
- Review: 6 months

Step 5 — Verify and measure

After deploying the filter, the FINOPS-02 daily burn-rate alert should show the bucket returning to baseline within 24-48 hours. If it doesn't, the spike has a different root cause (volume from non-DEBUG levels) and the framework loops back to Step 1 with a sharper question.

The diagnose → frame → coordinate → implement → verify loop is this entry's practice rather than a Dynatrace procedure — adapt it to your team's actual change-management process.

> **Sources:** OpenPipeline (DT docs), Trace a cost spike to its root cause (DT docs) — typical ingest-spike causes include *"debug-level logging left on in production"*.

9. Worked Example — High-Cardinality Metric

Scenario: Metrics-Ingest billable usage is climbing. The FINOPS-01 §8 query shows `monitoring_source == "other"` (custom metrics, with no included quota) dominating. One specific custom metric, `app.request.processed`, is contributing 70% of the volume.

Step 1 — Diagnose the cardinality

Query the metric's dimensions to find the high-cardinality one:

```
// Cardinality audit — how many distinct series a metric is producing
//
// Corrected 08/12/2026: `describe` is a STANDALONE command taking a data
object
// (`describe metric.series`), not a pipeline stage — appending `| describe`
failed with
// "1 mandatory parameter is missing: dataObject". It also reports the SCHEMA,
not per-dimension
// cardinality, so it cannot answer this question. Count distinct dimension
values directly.
//
// For the scenario: filter metric.key == "app.request.processed" and count
distinct app.name, endpoint, request.id.
// Runnable demo of the same pattern on an ActiveGate self-monitoring metric:
fetch metric.series, from:-24h
| filter metric.key == "dt.sfm.active_gate.thread_pool.queue_size"
| summarize {
    series          = count(),
    distinct_gates  = countDistinctExact(dt.active_gate.id),
    distinct_pools  = countDistinctExact(thread_pool_name)
  }

// Read it as: `series` is the metric's cardinality. A dimension whose
distinct count runs to
// thousands or millions is the one multiplying your series count — that is
the cost driver.
```

Suppose the result shows: `app.name` (12 values), `endpoint` (47 values), `request.id` (8.4M values). `request.id` is the runaway dimension — every request creates a new time series.

Step 2 — Frame the levers

- **Cut?** The metric itself (`app.request.processed`) is useful — it's the count of processed requests. Cut would lose load-bearing data.
- **Tune?** Lengthening the ingest interval helps proportionally but doesn't address the cardinality root cause.
- **Filter?** Drop the `request.id` dimension at the SDK or at ingest. The metric remains; the cardinality explosion stops.

Filter is the right lever. Specifically, remove the `request.id` label from the metric emission.

Step 3 — Coordinate before changing

`request.id` was added for some reason. The application team may have used it for per-request debugging via traces or logs (where high cardinality is appropriate). If yes — confirm the use case is satisfied via spans / logs, not metrics. If no — the label can be removed.

Step 4 — Implement

Implementation lives in the application's metric-emission code (OneAgent SDK, OpenTelemetry SDK, etc.). The framework-level spec:

- Source: application emitting `app.request.processed`
- Change: remove `request.id` label
- Owner: app team
- Review: not needed — root-cause fix

Step 5 — Verify

After the app team deploys the change, the metric's cardinality should drop to the cross-product of remaining dimensions (`app.name` × `endpoint` = ~564 series instead of 8.4M+). Billable data_points should drop accordingly.

The pattern

High-cardinality metrics are a documented cost driver — Dynatrace's guidance names user IDs and request IDs, and advises avoiding volatile dimensions when creating custom metrics. In community practice the root cause is usually a label that belongs on a span or log record rather than a metric dimension. The fix is structural — emit it as a span attribute (where high cardinality is fine) and remove it from the metric.

> **Sources:** DPS Metrics (DT docs). Best practices for optimizing metrics cost (DT docs) — *"High-cardinality dimensions, such as user IDs or request IDs, significantly increase metric storage requirements. Removing a high-cardinality dimension reduces the number of stored timeseries and can lower cost."*; *"avoid using volatile dimensions"*. The demo cardinality query executed on a SaaS tenant 10/05/2026 (136 series, 5 ActiveGates, 17 thread pools; no notifications).

10. Cross-References to Implementation Depth

This entry is the *framework* — it answers *which lever, when, why*. The implementation depth (how to write the OpenPipeline filter, how to configure retention per bucket, how to tune sample rate) lives in adjacent series:

For depth on...	See
Bucket strategy and retention configuration	**ORGNZ** topic series
OpenPipeline filter syntax for logs	**OPLOGS** topic series
OpenPipeline for non-log signals (spans, metrics, events)	**OPIPE** topic series
Trace sampling configuration	**SPANS** topic series
Custom metric emission with OneAgent SDK or OpenTelemetry	**OTEL** topic series
Synthetic monitor configuration and frequency tuning	**SYNTH** topic series
Optimization in the maturity-roadmap context	**ADOPT-05** notebook
`dt.cost.costcenter` / `dt.cost.product` label setup at ingest	**FAQ-02** (tagging sources, standards, strategy)

This entry is intentionally shallow on syntax — the goal is to structure the decision before the team reaches for the implementation depth in the adjacent series. Reach for the right *lever* first; then read the adjacent series for the *handle*.

> **Sources:** Cross-reference targets are the project's existing topic series. Implementation depth is owned in those series, not duplicated here.

11. Recommended Approach

A repeatable workflow for using the three-lever framework when an alert (FINOPS-02) fires:

1. **Diagnose at the bucket / capability / dimension level.** Use the FINOPS-01 §6-§10 queries to pinpoint *where* the consumption is. "Logs are up" is not enough — *which*

bucket, which loglevel, which app.

2. **Frame the lever before reaching for syntax.** Ask the three diagnostic questions: is this signal load-bearing (Cut?), is the parameter wrong (Tune?), is a subset noisy (Filter?). Pick one lever; reaching for two at once muddies the diagnosis.
3. **Reach for the upstream-most lever that solves the problem.** Cut at source > Filter at OpenPipeline > Tune at storage. Each step downstream is cheaper to implement but moves further from the root cause.
4. **Coordinate before implementing.** The team that owns the workload should be informed (Cut, Filter) or consulted (Tune on their signal). Cost optimization done unilaterally erodes trust faster than it saves money.
5. **Implement, verify, document.** The FINOPS-02 burn-rate alert is also the verification mechanism — if the change worked, the alert should clear within 24-48 hours. Document the change with owner, rationale, review date.
6. **Set the review cadence.** Every Cut and every Filter should have a documented review date (6 months default). Reviews catch the case where the optimization was correct at the time and isn't anymore.
7. **Resist the urge to combine optimizations across teams in one change.** One team, one lever, one change at a time. Bundled cost reductions become un-debuggable when something breaks.

12. Summary

Every DPS consumption-reduction option reduces to one of three levers: Cut (stop emitting), Tune (adjust a parameter), or Filter (drop a subset). Match the lever to the question — *is the signal load-bearing*, *is the parameter wrong*, *is a subset noisy*. Bias toward upstream levers — emission-time changes beat ingest-time changes beat storage-time changes. Coordinate before changing; document with owner, rationale, and review date. The cost savings that hurt most are the ones that surface six months later during an incident, because someone optimized a signal that turned out to be load-bearing — mitigate by treating signals that participate in incident response as off-limits without incident-team sign-off.

Next Steps

- Read **FINOPS-01** if you skipped it — the *what is being consumed* picture is the diagnostic foundation for the levers in this entry.

- Read **FINOPS-02** for the alert-and-forecast layer that signals *when* a lever needs to be reached for.
- Read **ORGNZ** for bucket strategy and retention configuration — the structural decisions that make Tune effective.
- Read **OPLOGS** for log-pipeline filter syntax — the implementation depth for the Filter lever on logs.
- Read **OPIPE** for OpenPipeline beyond logs — Filter implementations for spans, metrics, and events.
- Audit your tenant's most-expensive bucket against the §8 worked example. Apply the diagnose → frame → coordinate → implement → verify loop once; build the muscle.

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.